

CE 310 — Week 15 Final Exam Study Guide

Covers: Weeks 9–14 **Exam format:** Mon Nov 30 = Final Written Methods Exam (closed notes, closed book, closed computer and AI, 50 min, 50 pts); Wed Dec 2 = Python Practicum (open Co-lab/notes, no internet or AI, 50 min, 100 pts)

All numbers below are computed from the DOE Reference Medium Office dataset (Tucson, 8,760 hourly rows) used throughout Weeks 9–14, unless otherwise noted.

Week 9 — Simple Regression

Core concept. A simple linear regression fits a straight line $\hat{y} = \beta_0 + \beta_1 x$ that minimizes the sum of squared residuals. The slope β_1 is the predicted change in y per unit change in x . r^2 measures the fraction of variance in y explained by x .

Key Python functions

Function	What it does
<code>scipy.stats.linregress(x, y)</code>	Returns slope, intercept, r , p -value, <code>stderr</code>
<code>statsmodels.formula.api.ols('y ~ x', data).fit()</code>	Full regression object with <code>.summary()</code> , <code>.params</code> , <code>.rsquared</code>

Worked example. Cooling_kWh ~ Outdoor_Temp_F: slope = 1.1585 kWh/°F, intercept = -60.68, $r^2 = 0.601$, $p \approx 0$. “For every 1°F rise in outdoor temperature, predicted cooling demand increases by 1.16 kWh/hr.” At the ASHRAE 1% design temperature (108°F), predicted cooling = 64.44 kWh/hr — but the **95% prediction interval** for a single new hour is (30, 99) kWh/hr, far wider than the 95% CI on the mean prediction (63.6, 65.2) kWh/hr.

Common mistake. Confusing a confidence interval (uncertainty in the *average* prediction) with a prediction interval (uncertainty in a *single new observation*). The PI is always wider — it includes both the model’s uncertainty about the mean AND the natural scatter of individual points around that mean.

Extrapolation. A fitted model describes only the range it was fitted on. The Tucson data spans roughly **27°F to 106°F**, so a prediction at 118°F — or at 0°F, which is what the intercept -60.68 literally reports — is unsupported. Nothing in the fit says the relationship stays linear outside the observed window, and physically it cannot: chiller capacity saturates at the top end, so the true curve must bend. **Be ready to:** decline an out-of-range prediction and give a specific reason for declining.

Week 10 — Multiple Regression

Core concept. Adding predictors can change the coefficient on an existing predictor if the new predictor is correlated with both x and y (omitted variable bias). Interaction terms let the slope on one variable depend on the value of another.

Key Python functions

Function	What it does
<code>ols('y ~ x1 + x2', data).fit()</code>	Additive multiple regression
<code>ols('y ~ x1 * x2', data).fit()</code>	Includes x1, x2, AND their interaction term
<code>.rsquared_adj</code>	Adjusted R ² — penalizes adding predictors that don't help

Worked example. Model 2 (Cooling_kWh ~ Outdoor_Temp_F + BH): intercept = -56.87, Outdoor_Temp_F coefficient = **1.026**, BH coefficient = **17.85**, R² = 0.697. Note the Outdoor_Temp_F slope *dropped* from Week 9's 1.1585 to 1.026 — some of what looked like a pure temperature effect was really occupancy (BH), which Week 9's simple model couldn't separate out.

Model 5 (interaction, Cooling_kWh ~ Outdoor_Temp_F * BH): unoccupied slope = 0.7127 kWh/°F, occupied slope = 0.7127 + 1.0687 = **1.7813 kWh/°F** — occupied hours are 2.5× more temperature-sensitive than unoccupied hours, because internal/occupancy heat gains compound with outdoor temperature during operating hours.

Common mistake. Assuming a coefficient's value or sign is fixed and “true” regardless of what else is in the model. A coefficient is only interpretable *relative to the other predictors present* — adding or removing a correlated predictor can change it.

R² vs. adjusted R². Plain R² can **never decrease** when you add a predictor, even a column of pure noise — so a rise in R² is not by itself evidence that the new predictor helps. Adjusted R² charges a penalty for each parameter spent. When R² rises a little but adjusted R² *falls*, the predictor has not earned its place and should come out. **Be ready to:** read that pair of numbers and say what it means.

Week 11 — Hypothesis Testing

Core concept. A two-sample t-test asks whether two group means differ by more than chance would predict. The p-value is the probability of seeing a difference this large (or larger) if the true means were equal. Statistical significance (small p) is different from practical/engineering significance (is the difference big enough to matter?) — use effect size (Cohen's d) to judge the latter.

Key Python functions

Function	What it does
<code>scipy.stats.ttest_ind(group1, group2)</code>	Two-sample t-test, returns t-statistic and p-value
<code>scipy.stats.f_oneway(g1, g2, g3, ...)</code>	One-way ANOVA across 3+ groups
<code>Cohen's d = (mean₁ - mean₂) / pooled_std</code>	Standardized effect size

Worked example. Occupied-hours cooling (mean = 38.86 kWh/hr, n = 2,600) vs. unoccupied (mean = 10.56 kWh/hr, n = 6,160): mean difference = 28.30 kWh/hr, t = 55.68, p ≈ 0, Cohen's d = 1.30 (a **large** effect by conventional thresholds — d > 0.8). Season also matters: one-way ANOVA across 4 seasons gives F = 908.79, p ≈ 0 (Summer mean 38.41 vs. Winter mean 4.45 kWh/hr).

Common mistake. Reporting “p ≈ 0, therefore the effect is huge and important.” A tiny p-value only means the difference is unlikely to be due to random chance — with a large enough sample (n =

8,760 here), even a trivially small difference can produce $p \approx 0$. Always check effect size (Cohen's d) alongside p -value before calling a result practically important.

When p is large — the other direction. Every worked result above has $p \approx 0$, but the exam will also ask you to read a result that is *not* significant. If $p = 0.21$ at $\alpha = 0.05$ you **fail to reject H_0** — which is not the same as proving H_0 true. The effect may be real but small, or the sample too small to detect it (a **Type II error**, failing to detect a real effect; a **Type I error** is the opposite, flagging an effect that isn't there). Two traps to avoid: p is *not* the probability that H_0 is true, and α is fixed **before** you see the result — never raised afterwards to make a finding significant. **Be ready to:** interpret a non-significant result correctly.

Week 12 — Sensitivity Analysis

Core concept. One-at-a-time (OAT) sensitivity analysis perturbs one input (or model parameter) at a time, holding others fixed, and measures the resulting change in output (“swing”). A tornado chart ranks parameters by swing size, largest at top, to show which input the output is most sensitive to.

Key Python functions

Function	What it does
<code>matplotlib.pyplot.barh(...)</code>	Horizontal bar chart — standard way to draw a tornado diagram
Manual OAT loop	Recompute model output at $\text{param} \times 1.10$ and $\text{param} \times 0.90$, take the difference

Worked example. At $\text{Outdoor_Temp_F} = 85^\circ\text{F}$, $\text{BH} = 1$, Model 2 baseline prediction = 48.19 kWh/hr. Perturbing each parameter $\pm 10\%$: Outdoor_Temp_F coefficient swing = 17.44 kWh/hr (~36% of baseline) — the largest; Intercept swing = 11.37 kWh/hr (~24%); BH coefficient swing = 3.57 kWh/hr (~7%). **The temperature coefficient is by far the most sensitive parameter.** Separately, a direct $+5^\circ\text{F}$ annual bias increases annual cooling energy by 27.05%, while a ± 260 -occupied-hour schedule shift changes it by only 2.8% — temperature sensitivity is **9.7× larger** than occupancy-schedule sensitivity.

What OAT misses. Because it moves one input at a time and returns the others to baseline, a one-at-a-time analysis cannot see effects that only appear when **two inputs move together** — exactly the interaction that Week 10's $x_1 * x_2$ term captures. OAT still gives you each input's direction, each input's individual magnitude, and the ranking between them; the blind spot is combined movement. **Be ready to:** name that limitation.

Common mistake. Treating all inputs as equally worth monitoring or controlling. A tornado chart exists precisely to focus engineering/operational attention on the one or two inputs that actually drive most of the output uncertainty — spending equal effort tracking a low-impact input (like occupancy scheduling here) is a poor use of resources compared to the dominant driver (temperature).

Week 13 — Monte Carlo Simulation

Core concept. Instead of one deterministic estimate, Monte Carlo simulation draws many random scenarios from input probability distributions, recomputes the output for each, and reports the re-

sulting output distribution (e.g., P10/P50/P90) instead of a single number. This turns “what will it cost?” into “what’s the range, and how likely is each part of that range?”

Key Python functions

Function	What it does
<code>numpy.random.default_rng(seed)</code>	Reproducible random number generator
<code>rng.normal(mean, std, size=N)</code>	Draw N samples from a Normal distribution
<code>numpy.percentile(results, [10, 50, 90])</code>	Compute P10/P50/P90 from simulated results

Worked example. N = 10,000 scenarios, seed = 42. Three inputs randomized: Δ Temperature ~ Normal(0, 2°F), Δ Occupied-hours ~ Normal(0, 100 hrs), electricity rate ~ Normal(\$0.12, \$0.01)/kWh. Deterministic baseline annual cost = \$19,932.8. Simulated: mean = \$19,915.7, **P10 = \$16,468.6, P90 = \$23,488.4** (an \$7,019.8 spread — about 17.6% of the mean). Variance decomposition shows temperature drives 62.9% of the cost uncertainty, electricity rate 36.4%, and occupancy only 0.62%.

Why N = 10,000? Accuracy depends directly on the number of runs — the standard error of any simulated quantity falls as $1/\sqrt{N}$. The **mean converges fast** and is usable at small N; the **tails do not**. At N = 200, P90 rests on only about 20 samples, so it is noisy and visibly changes when you change the seed. At N = 10,000 it rests on about 1,000 and settles down. A smaller N does not bias the answer high or low, and does not systematically narrow the P10–P90 range — it just makes the tail estimates unstable in both directions. **Be ready to:** say what degrades when N is cut, and what does not.

Common mistake. Reporting only the mean of the Monte Carlo output and discarding the spread. The entire value of Monte Carlo simulation is the *distribution* — for budgeting or risk decisions, P10/P90 (or a full histogram) tells you far more than a single mean number, which by itself looks no different from the deterministic estimate.

Week 14 — Bootstrap Confidence Intervals

Core concept. Bootstrap resampling estimates the sampling distribution of a statistic (mean, slope, percentile, ...) by resampling the *observed* data with replacement, many times, and computing the statistic on each resample. This works without assuming a specific underlying distribution, and can reveal when the data violates assumptions (e.g., independence) that analytical formulas rely on.

Key Python functions

Function	What it does
<code>rng.choice(data, size=n, replace=True)</code>	One bootstrap resample, drawn with replacement
Loop N_BOOT times, recompute the statistic each time	Builds the bootstrap distribution
<code>numpy.percentile(boot_stats, [2.5, 97.5])</code>	95% percentile bootstrap CI

Worked example. N_BOOT = 1,000, seed = 42. Bootstrap 95% CI for mean Outdoor_Temp_F: (68.409, 69.087)°F — nearly identical to the analytical CI ($\bar{x} \pm 1.96 \times SE = 68.74 \pm 0.354$), as expected

for a well-behaved statistic like the mean. But the bootstrap 95% CI for the **regression slope** (1.1320, 1.1814) is **24.9% wider** than the OLS analytical CI — because hourly energy data is autocorrelated (today's cooling load resembles yesterday's), which violates OLS's independence assumption and makes its analytical standard errors too small. The bootstrap, which resamples actual rows, better reflects the true uncertainty.

Common mistake. Assuming bootstrap and analytical (formula-based) confidence intervals will always closely agree. They agree well for simple statistics like the mean (by the Central Limit Theorem) but can diverge substantially for statistics like regression slopes, R^2 , or percentiles — especially when the data has structure (autocorrelation, skew) that the analytical formula's assumptions don't account for.

Formula / Concept Sheet

Week	Concept	Key numbers to remember
9	Simple regression	slope 1.1585, intercept -60.68, $R^2=0.601$
10	Multiple regression	Model 2: BH coef 17.85, $R^2=0.697$; Model 5: occupied slope 1.78 vs unoccupied 0.71
11	Hypothesis testing	$t=55.68$, $p \approx 0$, Cohen's $d=1.30$ (large effect); ANOVA $F=908.79$ across seasons
12	Sensitivity analysis	temp coefficient swing 17.44 (largest); temp sensitivity $9.7 \times$ occupancy sensitivity
13	Monte Carlo	$N=10,000$, seed=42; $P10=\$16,468.6$, $P90=\$23,488.4$
14	Bootstrap CI	$N_BOOT=1,000$, seed=42; slope CI 24.9% wider than analytical (autocorrelation)